

TUẦN 07 TRIGGER

USE AdventureWorks2008R2

GO

SELECT *

FROM Sales.SalesOrderHeader

GO

-- 1. Tạo một Instead of trigger thực hiện trên view. Thực hiện theo các bước sau:

-- Tạo mới 2 bảng M_Employees và M_Department theo cấu trúc sau:

-- create table M_Department

-- (

-- DepartmentID int not null primary key,

-- Name nvarchar(50),

-- GroupName nvarchar(50)

--)

-- create table M_Employees

-- (

-- EmployeeID int not null primary key,

-- Firstname nvarchar(50),

-- MiddleName nvarchar(50),

-- LastName nvarchar(50),

-- DepartmentID int foreign key references M_Department(DepartmentID)

--)

-- Tạo một view tên EmpDepart_View bao gồm các field: EmployeeID,

-- FirstName, MiddleName, LastName, DepartmentID, Name, GroupName, dựa

-- trên 2 bảng M_Employees và M_Department.

-- Tạo một trigger tên InsteadOf_Trigger thực hiện trên view

-- EmpDepart_View, dùng để chèn dữ liệu vào các bảng M_Employees và

-- M_Department khi chèn một record mới thông qua view EmpDepart_View.

-- Dữ liệu test:

-- insert EmpDepart_view values(1, 'Nguyen', 'Hoang', 'Huy', 11, 'Marketing', 'Sales')

-- tạo table

CREATE TABLE M_Department

(

DepartmentID INT NOT NULL PRIMARY KEY,

Name NVARCHAR(50),

GroupName NVARCHAR(50)

)

CREATE TABLE M_Employees

(

EmployeeID INT NOT NULL PRIMARY KEY,

Firstname NVARCHAR(50),

MiddleName NVARCHAR(50),

LastName NVARCHAR(50),

DepartmentID INT FOREIGN KEY REFERENCES M_Department(DepartmentID)

)

GO

-- tạo view

CREATE VIEW EmpDepart_View

AS

SELECT e.EmployeeID, e.FirstName, e.MiddleName, e.LastName,

d.DepartmentID, d.Name, d.GroupName

FROM M_Department AS d JOIN M_Employees AS e

ON d.DepartmentID = e.DepartmentID

GO

```

-- tạo trigger
CREATE TRIGGER insteadof_trigger ON EmpDepart_View
instead OF INSERT
AS
BEGIN
    INSERT M_Department
    SELECT DepartmentID, Name, groupName
    FROM inserted

    INSERT M_Employees
    SELECT EmployeeID, FirstName, MiddleName, LastName, DepartmentID
    FROM inserted
END
GO

-- test trigger
SELECT *
FROM EmpDepart_View
INSERT EmpDepart_view
VALUES(1, 'Nguyen', 'Hoang', 'Huy', 11, 'Marketing', 'Sales')

SELECT *
FROM M_Department
SELECT *
FROM M_Employees

DROP VIEW dbo.EmpDepart_View
DROP TRIGGER dbo.insteadof_trigger
DROP TABLE dbo.M_Department
DROP TABLE dbo.M_Employees

```

```

-- 2. Tạo một trigger thực hiện trên bảng MySalesOrders có chức năng thiết lập độ ưu
-- tiên của khách hàng (CustPriority) khi người dùng thực hiện các thao tác Insert,
-- Update và Delete trên bảng MySalesOrders theo điều kiện như sau:
-- Nếu tổng tiền Sum(SubTotal) của khách hàng dưới 10,000 $ thì độ ưu tiên của
-- khách hàng (CustPriority) là 3 Nếu tổng tiền Sum(SubTotal) của khách hàng từ 10,000 $ đến dưới 50000
-- $ thì độ ưu tiên của khách hàng (CustPriority) là 2 Nếu tổng tiền Sum(SubTotal) của khách hàng từ
-- 50000 $ trở lên thì độ ưu tiên
-- của khách hàng (CustPriority) là 1
-- Các bước thực hiện:
-- Tạo bảng MCustomers và MSalesOrders theo cấu trúc
-- sau: create table MCustomer
-- (
--         CustomerID int not null primary key,
--         CustPriority int
-- )
-- create table MSalesOrders
-- (
--         SalesOrderID int not null primary key,
--         OrderDate date,
--         SubTotal money, CustomerID int foreign key references MCustomer(CustomerID)
-- )
-- Chèn dữ liệu cho bảng MCustomers, lấy dữ liệu từ bảng Sales.Customer,
-- nhưng chỉ lấy CustomerID>30100 và CustomerID<30118, cột CustPriority cho
-- giá trị null.
-- Chèn dữ liệu cho bảng MSalesOrders, lấy dữ liệu từ bảng
-- Sales.SalesOrderHeader, chỉ lấy những hóa đơn của khách hàng có trong bảng
-- khách hàng.
-- Viết trigger để lấy dữ liệu từ 2 bảng inserted và deleted.
-- Viết câu lệnh kiểm tra việc thực thi của trigger vừa tạo bằng cách chèn thêm hoặc
-- xóa hoặc update một record trên bảng MSalesOrders

-- tạo table
CREATE TABLE MCustomer
(
    CustomerID INT NOT NULL PRIMARY KEY,
    CustPriority INT
)

CREATE TABLE MSalesOrders
(
    SalesOrderID INT NOT NULL PRIMARY KEY,
    OrderDate DATE,
    SubTotal MONEY,
    CustomerID INT FOREIGN KEY REFERENCES MCustomer(CustomerID)
)
GO

-- chèn dữ liệu
INSERT INTO MCustomer
    (CustomerID, CustPriority)
SELECT sc.CustomerID, NULL
FROM Sales.Customer AS sc
WHERE sc.CustomerID BETWEEN 30101 AND 30117

```

```

INSERT INTO MSalesOrders
    (SalesOrderID, OrderDate, SubTotal, CustomerID)
SELECT s.SalesOrderID, s.OrderDate, s.SubTotal, s.CustomerID
FROM Sales.SalesOrderHeader AS s
WHERE s.CustomerID BETWEEN 30101 AND 30117

SELECT *
FROM dbo.MSalesOrders
SELECT *
FROM dbo.MCustomer
GO

--tạo trigger CustPriority

CREATE TRIGGER set_CustPriority ON MSalesOrders
FOR INSERT, UPDATE, DELETE
AS
WITH
    CTE
    AS
    (
        SELECT CustomerId
        FROM inserted
        UNION
        SELECT CustomerId
        FROM deleted
    )
UPDATE Mcustomer
SET custpriority = (
CASE
WHEN t.Total < 10000 THEN 3
WHEN t.Total BETWEEN 10000 AND 49999 THEN 2
WHEN t.Total >= 50000 THEN 1
END
)
FROM MSalesOrders AS c INNER JOIN CTE ON CTE.CustomerId = c.CustomerId
LEFT JOIN (
SELECT MsalesOrders.customerID, SUM(SubTotal) Total
FROM MsalesOrders INNER JOIN CTE
ON CTE.CustomerId = MsalesOrders.CustomerId
GROUP BY MsalesOrders.customerID
) t ON CTE.CustomerId = t.CustomerId
GO

INSERT MsalesOrders
VALUES(71847, '2016-01-01', 10000, 30112)

SELECT*
FROM Mcustomer
WHERE CustomerId=30112

SELECT*
FROM MSalesOrders
GO

DROP TRIGGER dbo.set_CustPriority
DROP TABLE dbo.MSalesOrders
DROP TABLE dbo.MCustomer

```

```
-- 3. Viết một trigger thực hiện trên bảng MEmployees sao cho khi người dùng thực
-- hiện chèn thêm một nhân viên mới vào bảng MEmployees thì chương trình cập
-- nhật số nhân viên trong cột NumOfEmployee của bảng MDepartment. Nếu tổng
-- số nhân viên của phòng tương ứng <=200 thì cho phép chèn thêm, ngược lại thì
-- hiển thị thông báo “Bộ phận đã đủ nhân viên” và hủy giao tác. Các bước thực hiện:
-- Tạo mới 2 bảng MEmployees và MDepartment theo cấu trúc sau:
-- create table MDepartment
-- (
--     DepartmentID int not null primary key,
--     Name nvarchar(50),
--     NumOfEmployee int
-- )
-- create table MEmployees
-- (
--     EmployeeID int not null,
--     FirstName nvarchar(50),
--     MiddleName nvarchar(50),
--     LastName nvarchar(50),
--     DepartmentID int foreign key references MDepartment(DepartmentID),
--     constraint pk_emp_depart primary key(EmployeeID, DepartmentID)
-- )
-- Chèn dữ liệu cho bảng MDepartment, lấy dữ liệu từ bảng Department, cột
-- NumOfEmployee gán giá trị NULL, bảng MEmployees lấy từ bảng
-- EmployeeDepartmentHistory
-- Viết trigger theo yêu cầu trên và viết câu lệnh hiện thực trigger
```

```
CREATE TABLE MDepartment
```

```
(
    DepartmentID INT NOT NULL PRIMARY KEY,
    Name NVARCHAR(50),
    NumOfEmployee INT
)
```

```
CREATE TABLE MEmployees
```

```
(
    EmployeeID INT NOT NULL,
    FirstName NVARCHAR(50),
    MiddleName NVARCHAR(50),
    LastName NVARCHAR(50),
    DepartmentID INT FOREIGN KEY REFERENCES MDepartment(DepartmentID),
    CONSTRAINT pk_emp_depart PRIMARY KEY(EmployeeID, DepartmentID)
)
GO
```

```
INSERT MDepartment
```

```
SELECT [DepartmentID], [Name], NULL
FROM [HumanResources].[Department]
```

```
INSERT [MEmployees]
```

```
SELECT e.[BusinessEntityID], [FirstName], [MiddleName], [LastName], [DepartmentID]
FROM [HumanResources].[Employee] e JOIN [Person].[Person] p ON e.BusinessEntityID=p.BusinessEntityID
JOIN [HumanResources].[EmployeeDepartmentHistory] h ON e.BusinessEntityID=h.BusinessEntityID
```

```
SELECT*
```

```
FROM [dbo].[MEmployees]
ORDER BY [DepartmentID]
GO
```

```

-- tạo trigger
CREATE TRIGGER kiemTraSLNV ON dbo.MEmployees
FOR INSERT
AS
    DECLARE @slnv INT, @DepartID INT
    SELECT @DepartID = i.DepartmentID
FROM inserted AS i

    SET @slnv = (
        SELECT COUNT(*)
FROM [dbo].[MEmployees] e
WHERE e.DepartmentID = @DepartID
    )

    IF(@slnv > 200)
        BEGIN
            PRINT N'Bộ phận đã đủ nhân viên'
            ROLLBACK
        END
    ELSE
        BEGIN
            UPDATE MDepartment
            SET NumOfEmployee = @slnv
            WHERE DepartmentID = @DepartID
        END
GO

--test trigger
INSERT [dbo].[MEmployees]
VALUES(291, 'Nguyen', 'Hoang', 'Anh', 1),
      (292, 'Nguyen', 'Hoang', 'Thu', 2)

--kiem tra ket qua
SELECT *
FROM MDepartment

DROP TRIGGER dbo.kiemTraSLNV;
DROP TABLE dbo.MEmployees
DROP TABLE dbo.MDepartment
GO

```

```

-- 4. Bảng [Purchasing].[Vendor], chứa thông tin của nhà cung cấp, thuộc tính
-- CreditRating hiển thị thông tin đánh giá mức tín dụng, có các giá trị:
-- 1 = Superior
-- 2 = Excellent
-- 3 = Above average
-- 4 = Average
-- 5 = Below average
-- Viết một trigger nhằm đảm bảo khi chèn thêm một record mới vào bảng
-- [Purchasing].[PurchaseOrderHeader], nếu Vender có CreditRating=5 thì hiển thị
-- thông báo không cho phép chèn và đồng thời hủy giao tác.
-- Dữ liệu test
-- INSERT INTO Purchasing.PurchaseOrderHeader (RevisionNumber, Status,
-- EmployeeID, VendorID, ShipMethodID, OrderDate, ShipDate, SubTotal, TaxAmt,
-- Freight) VALUES ( 2 ,3, 261, 1652, 4 ,GETDATE() ,GETDATE() , 44594.55,
-- ,3567.564, ,1114.8638 );

```

```

CREATE TRIGGER Purchasing.CreditRating_trigger ON [Purchasing].[PurchaseOrderHeader]
FOR INSERT
AS
    IF EXISTS (
        SELECT *
FROM [Purchasing].[PurchaseOrderHeader] AS p JOIN inserted AS i
    ON p.PurchaseOrderID = i.PurchaseOrderID JOIN [Purchasing].[Vendor] AS v
    ON p.VendorID = v.BusinessEntityID
WHERE v.CreditRating = 5
    )
    BEGIN
        RAISERROR ('A vendor's credit rating is too low to accept new purchase orders.', 16, 1);
        ROLLBACK TRANSACTION
    END
GO

INSERT INTO Purchasing.PurchaseOrderHeader
    (RevisionNumber, Status, EmployeeID, VendorID, ShipMethodID, OrderDate, ShipDate, SubTotal,
    TaxAmt, Freight)
VALUES
    ( 2, 3, 261, 1652, 4 , GETDATE() , GETDATE() , 44594.55, 3567.564, 1114.8638 )

DROP TRIGGER Purchasing.CreditRating_trigger
GO

```

```
-- 5. Viết một trigger thực hiện trên bảng ProductInventory (lưu thông tin số lượng sản
-- phẩm trong kho). Khi chèn thêm một đơn đặt hàng vào bảng SalesOrderDetail với
-- số lượng xác định trong field
-- OrderQty, nếu số lượng trong kho
-- Quantity > OrderQty thì cập nhật
-- lại số lượng trong kho
-- Quantity= Quantity- OrderQty,
-- ngược lại nếu Quantity=0 thì xuất
-- thông báo “Kho hết hàng” và đồng
-- thời hủy giao tác.
```

```
-- tạo bảng MProduct
```

```
CREATE TABLE MProduct
(
    MProductID INT NOT NULL PRIMARY KEY,
    ProductName NVARCHAR(50),
    ListPrice MONEY
)
```

```
INSERT MProduct
    (MProductID, ProductName, ListPrice)
SELECT [ProductID], [Name], [ListPrice]
FROM [Production].[Product]
WHERE [ProductID] <= 710
SELECT*
FROM MProduct
```

```
-- tạo bảng MSalesOrderHeader
```

```
CREATE TABLE MSalesOrderHeader
(
    MSalesOrderID INT NOT NULL PRIMARY KEY,
    OrderDate DATETIME
)
```

```
INSERT MSalesOrderHeader
SELECT [SalesOrderID], [OrderDate]
FROM [Sales].[SalesOrderHeader]
WHERE [SalesOrderID] IN (SELECT [SalesOrderID]
FROM [Sales].[SalesOrderDetail]
WHERE [ProductID] <= 710)
SELECT*
FROM MSalesOrderHeader
```

```
-- tạo bảng MSalesOrderDetail
```

```
CREATE TABLE MSalesOrderDetail
(
    SalesOrderDetailID INT IDENTITY(1,1) PRIMARY KEY,
    ProductID INT NOT NULL FOREIGN KEY(ProductID) REFERENCES MProduct(MProductID),
    SalesOrderID INT NOT NULL FOREIGN KEY (SalesOrderID) REFERENCES
MSalesOrderHeader(MSalesOrderID),
    OrderQty INT
)
INSERT MSalesOrderDetail
    (ProductID, SalesOrderID, OrderQty)
SELECT [ProductID], [SalesOrderID], [OrderQty]
FROM [Sales].[SalesOrderDetail]
WHERE [ProductID] IN (SELECT MProductID
FROM MProduct)
```



```

-- tạo bảng MProduct_inventory
CREATE TABLE MProduct_inventory
(
    productID INT NOT NULL PRIMARY KEY,
    quantity SMALLINT
)

INSERT MProduct_inventory
SELECT [ProductID], sum([Quantity]) AS sumofquantity
FROM [Production].[ProductInventory]
GROUP BY [ProductID]
GO

-- tạo trigger
CREATE TRIGGER bai5 ON MSalesOrderDetail
FOR INSERT
AS
BEGIN
    DECLARE @Qty INT, @Quantity INT, @productID INT
    SELECT @qty = i.OrderQty, @productID = i.ProductID
    FROM inserted i

    SELECT @Quantity = p.Quantity
    FROM MProduct_inventory p
    WHERE @productID = p.ProductID

    IF(@Quantity > @Qty)
    BEGIN
        UPDATE MProduct_inventory
            SET quantity = @Quantity - @Qty
            WHERE productID = @productID
    END
    ELSE IF (@Quantity = 0)
    BEGIN
        PRINT N'Kho hết hàng'
        ROLLBACK
    END
END
GO

SELECT*
FROM MSalesOrderDetail
SELECT*
FROM MSalesOrderHeader
SELECT*
FROM MProduct_inventory
WHERE [ProductID]=708

---thuc thi trigger
DELETE FROM [MSalesOrderDetail]
INSERT [dbo].[MSalesOrderDetail]
VALUES(708, 43661, 300)

DROP TRIGGER bai5
DROP TABLE [MSalesOrderDetail]
DROP TABLE [MSalesOrderHeader]
DROP TABLE [MProduct_inventory]

```

```
-- 6. Tạo trigger cập nhật tiền thưởng (Bonus) cho nhân viên bán hàng SalesPerson, khi
-- người dùng chèn thêm một record mới trên bảng SalesOrderHeader, theo quy định
-- như sau: Nếu tổng tiền bán được của nhân viên có hóa đơn mới nhập vào bảng
-- SalesOrderHeader có giá trị >10000000 thì tăng tiền thưởng lên 10% của mức
-- thưởng hiện tại. Cách thực hiện:
-- Tạo hai bảng mới M_SalesPerson và M_SalesOrderHeader
-- create table M_SalesPerson
-- (
--     SalePSID int not null primary key,
--     TerritoryID int,
--     BonusPS money
-- )
-- create table M_SalesOrderHeader
-- (
--     SalesOrdID int not null primary key,
--     OrderDate date,
--     SubTotalOrd money,
--     SalePSID int foreign key references M_SalesPerson(SalePSID)
-- )
-- - Chèn dữ liệu cho hai bảng trên lấy từ SalesPerson và SalesOrderHeader chọn
-- những field tương ứng với 2 bảng mới tạo.
-- - Viết trigger cho thao tác insert trên bảng M_SalesOrderHeader, khi trigger
-- thực thi thì dữ liệu trong bảng M_SalesPerson được cập nhật.
```

```
CREATE TABLE M_SalesPerson
```

```
(
    SalePSID INT NOT NULL PRIMARY KEY,
    TerritoryID INT,
    BonusPS MONEY
)
```

```
CREATE TABLE M_SalesOrderHeader
```

```
(
    SalesOrdID INT NOT NULL PRIMARY KEY,
    OrderDate DATE,
    SubTotalOrd MONEY,
    SalePSID INT FOREIGN KEY REFERENCES M_SalesPerson(SalePSID)
)
```

```
INSERT INTO M_SalesPerson
```

```
SELECT s.BusinessEntityID, s.TerritoryID, s.Bonus
FROM Sales.SalesPerson AS s
```

```
INSERT INTO M_SalesOrderHeader
```

```
SELECT s.SalesOrderID, s.OrderDate, s.SubTotal, s.SalesPersonID
FROM Sales.SalesOrderHeader AS s
```

```
GO
```

```

CREATE TRIGGER bonus_trigger ON M_SalesOrderHeader
FOR INSERT
AS
    BEGIN
    DECLARE @doanhThu FLOAT, @maNV INT
    SELECT @maNV= i.SalePSID
    FROM inserted i

    SET @doanhThu=(
        SELECT sum([SubTotalOrd])
    FROM [dbo].[M_SalesOrderHeader]
    WHERE SalePSID=@maNV
    )

    IF (@doanhThu > 10000000)
    BEGIN
        UPDATE M_SalesPerson
            SET BonusPS += BonusPS * 0.1
            WHERE SalePSID=@maNV
    END
END
GO

```